UNIVERSIDADE DE SÃO PAULO (USP)

Instituto de Ciências Matemáticas e de Computação (ICMC)

Fundação de Apoio à Universidade de São Paulo (FUSP) — Projeto 605001

RELATÓRIO TÉCNICO DE ARQUITETURA E DESENVOLVIMENTO DE SOFTWARE

Sistema Web de Ponto Eletrônico, Banco de Horas e Emissão Automatizada do Boletim de Frequência FUSP

Estagiário / Autor:	Carlos Henrique da Luz Pizzi
Supervisão:	Maria Fernanda Marreta
Unidade / Setor:	ICMC / USP — Projeto FUSP nº 605001
Aplicação em Nuvem:	https://ponto-estagio-carlos.streamlit.app
Link da Supervisão:	https://ponto-estagio-carlos.streamlit.app/?acesso=gestao
Repositório Git:	https://github.com/carlinpz/ponto-estagio-icmc
Diretório Local (PC):	C:\Users\16913193\.gemini\antigravity\scratch\ponto_estagio
Legislação Base:	Lei Federal nº 11.788/2008 (Lei do Estágio)

São Carlos — SP

16 de Outubro de 2026

Sumário

1 Visão Geral e Objetivos do Projeto **2**
1.1 Principais Entregas e Benefícios . . . 2

2 Arquitetura, Armazenamento e Hospedagem em Nuvem **2**
2.1 Onde o Sistema Está Salvo? . . . 2
2.2 Fluxo de Comunicação e Sincronização de Dados . . . 3
2.3 Estabilidade, Prazos, Custos e "Modo Soneca" . . . 3

3 Tecnologias e Linguagens Utilizadas **4**

4 Engenharia de Software e Estrutura Modular do Código **5**
4.1 `app.py` — Orquestrador Principal e Controle de Sessão . . . 5
4.2 `storage.py` — Camada de Persistência Híbrida e Segurança de Dados . . . 5
4.3 `calculations.py` — Motor Matemático, Fuso Horário e Boletim FUSP . . . 6
4.4 `components.py` — Componentes Visuais, Login e Visão da Supervisão . . . 6
4.5 `theme.py` — Identidade Visual Metálica e Scripts Client-Side . . . 6
4.6 `test_app.py` — Suíte de Testes Automatizados . . . 7

5 Modelagem Matemática e Regras de Negócio **7**
5.1 Cálculo de Horas Líquidas Diárias . . . 7
5.2 Saldo Diário e Banco de Horas Acumulado . . . 7
5.3 Regra de Período do Boletim FUSP . . . 7

6 Auditoria de Qualidade e Correções Implementadas **7**

7 Guia Rápido de Operação e Credenciais **9**
7.1 Rotina Mensal de Fechamento FUSP . . . 9

1. Visão Geral e Objetivos do Projeto

O **Sistema Integrado de Ponto Eletrônico e Gestão de Estágio (ICMC • USP)** foi projetado e desenvolvido para substituir controles manuais em planilhas isoladas por uma aplicação web moderna, auditável e sincronizada em tempo real na nuvem.

O sistema atende simultaneamente às necessidades operacionais do estagiário (**Carlos Henrique da Luz Pizzi**) e às demandas de acompanhamento e validação da supervisão (**Maria Fernanda Marreta**), garantindo conformidade estrita com a **Lei Federal nº 11.788/2008 (Lei do Estágio)** e com o fluxo documental da **Fundação de Apoio à Universidade de São Paulo (FUSP)** no âmbito do projeto **605001**.

1.1 Principais Entregas e Benefícios

- **Registro de Ponto em Tempo Real:** Marcação de entrada, início de intervalo, retorno de intervalo e saída com relógio digital ao vivo sincronizado no fuso horário oficial de Brasília (UTC − 3).
- **Cálculo Automatizado de Banco de Horas:** Apuração instantânea de horas líquidas trabalhadas, desconto de intervalos intrajornada, comparação com a meta contratual diária (6, 0h/dia ou 30h/semana) e saldo acumulado ao longo dos meses.
- **Geração Automática do Boletim Oficial FUSP:** Emissão em 1 clique do documento oficial da FUSP no ciclo regulamentar do **dia 21 do mês anterior ao dia 20 do mês vigente**, pronto para impressão ou exportação em PDF para entrega até o dia 22.
- **Controle de Acesso por Perfil:** Separação entre o perfil operacional do estagiário (protegido por login com **Usuário e Senha** e token criptográfico de sessão) e o perfil executivo da supervisão (acesso direto em modo **Somente Leitura**).
- **Persistência Híbrida e Redundante:** Sincronização automática entre arquivo local (`ponto_eletronico.json`) e banco de dados em nuvem no **Google Sheets** via **Google Apps Script**.

2. Arquitetura, Armazenamento e Hospedagem em Nuvem

Para garantir disponibilidade contínua (24/7) a partir de qualquer dispositivo (computador pessoal, estação de trabalho na USP ou smartphone) sem custos de infraestrutura, o sistema utiliza uma arquitetura distribuída em três camadas integradas.

2.1 Onde o Sistema Está Salvo?

1. **No Computador Local (Ambiente de Desenvolvimento e Backup):**

   ```
   C:\Users\16913193\.gemini\antigravity\scratch\ponto_estagio\
   ```

   Nesta pasta residem os módulos em Python (`app.py`, `storage.py`, `calculations.py`, `components.py`, `theme.py`), a suíte de testes (`test_app.py`) e o banco de dados local espelhado (`ponto_eletronico.json`).

2. **No GitHub (Servidor de Código e Versionamento):**

```
https://github.com/carlinpz/ponto-estagio-icmc
```

Guarda todo o histórico de versões do código-fonte nas ramificações (*branches*) `main` e `master`. Qualquer atualização enviada via `git push` dispara automaticamente a atualização do site em produção.

3. **No Streamlit Community Cloud (Servidor de Hospedagem Web):**

```
https://ponto-estagio-carlos.streamlit.app
```

É o servidor em nuvem que executa o código Python e entrega a interface visual no navegador para o estagiário e para a supervisão.

4. **No Google Drive / Google Sheets (Banco de Dados Permanente em Nuvem):**

```
https://docs.google.com/spreadsheets/d/1-1XDAVUDdAGK-OUfCog8bDizN8Ax-oeHfl81rHlxfmI
```

Como servidores em nuvem utilizam contêineres efêmeros (que reiniciam periodicamente), a planilha do Google Drive atua como a **fonte única da verdade (*Single Source of Truth*)**, armazenando tanto os registros diários de ponto quanto as configurações e credenciais de acesso.

2.2 Fluxo de Comunicação e Sincronização de Dados

O ciclo de leitura e gravação foi otimizado para oferecer velocidade de carregamento inferior a meio segundo:

Fluxo Operacional de Dados (Leitura Rápida + Escrita Transacional)

1. **Leitura Instantânea (CSV Export):** Ao abrir o aplicativo, o módulo `storage.py` requisita o endpoint de exportação CSV da planilha Google com um parâmetro *cache-buster* (`&t=timestamp`). Os registros e a linha especial `CONFIG_JSON` são carregados em memória na sessão (`st.session_state.db`).
2. **Processamento em Memória:** Durante a navegação entre abas e filtros, o sistema utiliza o cache em sessão para evitar requisições repetitivas na rede, garantindo resposta imediata a cada clique.
3. **Gravação Segura (Webhook Apps Script):** Sempre que um ponto é batido, um ajuste manual é salvo ou a senha é alterada, a função `save_database()` grava o JSON local e dispara uma requisição HTTP `POST` para o Web App do **Google Apps Script**, que atualiza imediatamente a planilha na nuvem.

2.3 Estabilidade, Prazos, Custos e "Modo Soneca"

- **Custo e Prazo de Validade:** Todos os serviços utilizados (GitHub, Streamlit Community Cloud, Google Drive e Google Apps Script) operam em planos gratuitos permanentes (*Free Tier*), **sem prazo de expiração** e sem cobrança de mensalidade.

- **Segurança contra Perda de Dados:** Mesmo que o servidor do Streamlit passe por manutenção ou reinicialização, **nenhum dado é perdido**, pois todas as marcações e credenciais estão gravadas diretamente na sua planilha do Google Drive.
- **Comportamento de Hibernação (Modo Soneca):** O Streamlit Community Cloud possui uma política de economia de recursos na qual aplicativos que ficam **de 5 a 7 dias consecutivos sem nenhum acesso** entram em modo de espera (*sleep mode*).
  - Como o ponto é utilizado diariamente de segunda a sexta-feira, o aplicativo permanecerá sempre ativo.
  - Caso ocorra um período longo de férias/recesso e o app entre em hibernação, ao abrir o link será exibido um botão azul **"Yes, get this app back up!"**. Basta um clique para o servidor religar em cerca de 20 segundos, restaurando automaticamente todos os dados da planilha.

3. Tecnologias e Linguagens Utilizadas

Embora o projeto seja centrado em Python, ele combina cinco tecnologias complementares para entregar uma experiência visual e funcional de alto nível:

Linguagem / Tecnologia	Proporção	Papel no Sistema
Python 3.11+	$\sim 80\%$	Lógica central de negócios, cálculos de jornada, manipulação de datas/fusos, integração HTTP com nuvem e estrutura de dados.
Streamlit	Framework UI	Renderização reativa da interface web, gerenciamento de sessão, formulários, autenticação e hospedagem em nuvem.
Pandas & Plotly	Dados / Gráficos	Estruturação tabular de séries temporais, cálculo vetorial de saldos acumulados e gráficos interativos de desempenho.
HTML5 & CSS3	$\sim 12\%$	Estilização metálica inspirada no ICMC/USP (*glassmorphism*), cartões de indicadores e diagramação fiel do Boletim FUSP A4.
JavaScript (ES6+)	$\sim 8\%$	Animação física em `<canvas>` 60fps (rede neural interativa no fundo), relógio digital em tempo real no DOM e webhook do Google Apps Script.
JSON & CSV	Dados	Protocolos de serialização para persistência local (`.json`), leitura rápida da nuvem e exportação compatível com Excel (`.csv` UTF-8 BOM).

Tabela 1: Stack tecnológico empregado no projeto.

4. Engenharia de Software e Estrutura Modular do Código

O código-fonte foi arquitetado seguindo o princípio de **Separação de Responsabilidades (*Separation of Concerns*)**, dividindo o sistema em 6 arquivos independentes e coesos.

4.1 `app.py` — Orquestrador Principal e Controle de Sessão

É o ponto de entrada (*entry point*) executado pelo comando `streamlit run app.py`. Suas responsabilidades incluem:

- Inicializar e manter em cache o banco de dados na sessão (`init_session_state`), evitando chamadas de rede desnecessárias a cada clique.
- Validar o perfil de acesso ativo (`estagiario` vs. `gestao`) e verificar o token criptográfico na URL (`?acesso=estagiario&token=...`) para manter a sessão autenticada ao atualizar a página (F5).
- Renderizar a barra lateral com seletor de período (mês/ano), botão de sincronização manual com a nuvem (**Sincronizar Nuvem**), painel de parâmetros contratuais e botões de ponto rápido.
- Organizar as 6 abas operacionais da visão do estagiário:
  1. *Registro Rápido*
  2. *Gráficos & Análise*
  3. *Histórico Mensal*
  4. *Ajuste Manual* (com suporte a expedientes em andamento e feriados/abonos)
  5. *Boletim FUSP*
  6. *Exportação & Backup*

4.2 `storage.py` — Camada de Persistência Híbrida e Segurança de Dados

Gerencia toda a entrada e saída de dados entre o disco local e o Google Drive:

- `load_database_from_sheet_csv()`: Faz o download instantâneo da planilha Google em formato CSV, converte horários e datas com expressões regulares (`_clean_cloud_time` e `_clean_cloud_date`) e extrai o objeto JSON de configurações salvo na linha `CONFIG_JSON`.
- `save_database_to_apps_script()`: Empacota todos os registros diários junto com a linha especial `CONFIG_JSON` (que guarda nome, supervisora, projeto, usuário e senha) e envia via `POST` para o webhook do Google Apps Script.
- `_deduplicate_records()`: Garante unicidade absoluta por data (`YYYY-MM-DD`), impedindo que cliques simultâneos ou retransmissões de rede gerem linhas duplicadas nas tabelas ou relatórios.
- `_merge_configs()`: Impede que leituras parciais ou valores vazios sobrescrevam o usuário e a senha cadastrados pelo estagiário.
- `_parse_meta_dia()`: Preserva metas iguais a `0.0` em dias de feriado, recesso ou abono.

4.3 `calculations.py` — Motor Matemático, Fuso Horário e Boletim FUSP

Concentra todas as regras de cálculo trabalhista/acadêmico e geração documental:

- **Fuso Horário Oficial de Brasília (`BRT_TZ`):** Define `timezone(timedelta(hours=-3))` nas funções `now_br()` e `today_br()`. Como os servidores do Streamlit Cloud operam em horário `UTC` (+3h), essa camada garante que todas as batidas e cálculos de tempo decorrido reflitam exatamente o horário de São Carlos/Brasília.
- **Cálculo de Jornada (`calculate_day_worked_hours`):** Converte marcações `HH:MM:SS` em objetos `datetime`, subtrai o intervalo intrajornada e retorna as horas líquidas em formato decimal e o status do dia (*Concluído*, *Em andamento*, *Feriado / Abono* ou *Ausente*).
- **Auditoria da Lei do Estágio (`detect_compliance_issues`):** Inspeciona os registros em busca de marcações incompletas em dias anteriores (entrada sem saída ou pausa sem retorno) e jornadas acima do limite legal de 6h diárias da Lei 11.788/2008.
- **Gerador Oficial do Boletim FUSP (`generate_fusp_timesheet_html`):** Calcula o ciclo exato da FUSP (`get_fusp_period_dates`, do dia 21 do mês anterior ao dia 20 do mês selecionado) e constrói o HTML/CSS fiel ao modelo da Fundação, incluindo logotipo embutido em Base64, tabela de 30/31 dias, destaque para sábados/domingos e campos de assinatura do estagiário e da coordenação.

4.4 `components.py` — Componentes Visuais, Login e Visão da Supervisão

Encapsula os blocos de interface gráfica:

- `render_welcome_screen()`: Tela inicial de escolha de perfil com acesso direto para a Supervisão e formulário de login com **Usuário e Senha** para o estagiário, além de um painel expansível para alteração de credenciais sincronizado na nuvem.
- `compute_auth_token()`: Gera um hash criptográfico truncado (`SHA-256`) a partir do par `usuário:senha` para manter a sessão autenticada no navegador sem expor a senha na URL.
- `render_metric_cards()` e `render_charts()`: Renderizam os 4 cartões superiores de KPI e os 3 gráficos interativos em Plotly (Saldo Diário em barras, Evolução do Saldo Acumulado em linha e Horas Realizadas vs. Meta).
- `render_manager_screen()`: Painel executivo exclusivo da supervisão (**Maria Fernanda Marreta**), contendo status em tempo real do estagiário hoje, indicadores de assiduidade, auditoria de conformidade legal, espelho mensal e download/prévia direta do **Boletim de Frequência FUSP**.

4.5 `theme.py` — Identidade Visual Metálica e Scripts Client-Side

Define a estética institucional inspirada no ICMC/USP:

- Paleta escura metálica (*obsidian*, *steel blue* `#38BDF8` e *copper amber* `#F59E0B`) com tipografia técnica (*Plus Jakarta Sans* e *JetBrains Mono*).
- Emblema vetorial SVG do nó matemático estilizado (`get_knot_svg_html`).
- Script JavaScript que injeta um elemento `<canvas id='neural-bg-global'>` diretamente no `document.body`, desenhando uma rede de partículas interativas a 60 quadros por segundo, acompanhado do script de atualização contínua do relógio digital a cada 250ms.

4.6 `test_app.py` — Suíte de Testes Automatizados

Script de verificação contínua que executa 12 baterias de testes unitários e de integração, validando cálculos com intervalo, horas extras, horas negativas, conversão de horários, processamento de DataFrame, exportação CSV, contagem de dias úteis, detecção de inconformidades legais, geração do Boletim FUSP, tratamento de feriados (`meta_dia = 0.0`) e proteção de credenciais.

5. Modelagem Matemática e Regras de Negócio

5.1 Cálculo de Horas Líquidas Diárias

Seja um dia de trabalho d com horários registrados de entrada T_{ent}, início de pausa $T_{\text{ini_p}}$, fim de pausa $T_{\text{fim_p}}$ e saída T_{sai}. O tempo bruto de permanência ΔT_{bruto} e a duração do intervalo intrajornada ΔT_{pausa} são dados por:

$$\Delta T_{\text{bruto}} = T_{\text{sai}} - T_{\text{ent}}, \qquad \Delta T_{\text{pausa}} = \begin{cases} \max(0,\ T_{\text{fim_p}} - T_{\text{ini_p}}), & \text{se houver pausa registrada,} \\ 0, & \text{caso contrário.} \end{cases}$$

As horas líquidas trabalhadas em formato decimal $H_{\text{líq}}(d)$ (em horas) são obtidas por:

$$H_{\text{líq}}(d) = \operatorname{round}\left(\frac{\max(0,\ \Delta T_{\text{bruto}} - \Delta T_{\text{pausa}})}{3600\ \text{s}},\ 2 \right).$$

5.2 Saldo Diário e Banco de Horas Acumulado

Para cada dia d com meta contratual $M(d)$ (padrão $M(d) = 6,0\text{h}$ em dias úteis e $M(d) = 0,0\text{h}$ em feriados/abonos):

$$S(d) = \begin{cases} H_{\text{líq}}(d) - M(d), & \text{se o expediente estiver concluído,} \\ 0,0, & \text{se estiver em andamento ou for Feriado / Abono } (M(d) = 0), \\ -M(d), & \text{se for dia útil ausente sem justificativa.} \end{cases}$$

O saldo acumulado até o k-ésimo dia concluído da série histórica é:

$$S_{\text{acum}}(k) = \sum_{i=1}^{k} S(d_i).$$

5.3 Regra de Período do Boletim FUSP

Para um mês de referência $m \in \{1, \ldots, 12\}$ e ano A, o período oficial de apuração da FUSP compreende todas as datas d no intervalo fechado $[D_{\text{início}}, D_{\text{fim}}]$:

$$D_{\text{início}} = \begin{cases} 21/12/(A-1), & \text{se } m = 1, \\ 21/(m-1)/A, & \text{se } m \in \{2, \ldots, 12\}, \end{cases} \qquad D_{\text{fim}} = 20/m/A.$$

6. Auditoria de Qualidade e Correções Implementadas

Durante o checkup técnico completo realizado em Outubro de 2026, foram diagnosticadas e solucionadas as seguintes falhas estruturais:

Falha Identificada	Causa Raiz	Solução de Engenharia Aplicada
Reset constante de senha a cada acesso	A leitura CSV retornava `DEFAULT_CONFIG` com senha vazia e sobrescrevia o JSON local; o Apps Script não salvava o objeto `config`.	Criação da linha de metadados `CONFIG_JSON` na planilha Google e da função `_merge_configs()`, garantindo persistência definitiva de usuário e senha na nuvem.
Ausência de login por Usuário + Senha	O sistema solicitava apenas um PIN simples e permitia acesso via parâmetro de URL sem verificação de token.	Implementação de login completo com **Usuário e Senha**, gerenciamento de credenciais e validação por token hash `SHA-256` (`compute_auth_token`).
Diferença de +3h no Streamlit Cloud	Os servidores do Streamlit Cloud operam em fuso `UTC`, adiantando batidas e cálculos em 3 horas em relação ao Brasil.	Criação das funções `now_br()` e `today_br()` com fuso fixo `BRT_TZ` (UTC − 3) em todos os módulos.
Feriado (0h) tratado como falta (−6h)	Expressão booleana avaliava `meta_dia = 0.0` como `False`, revertendo a meta para 6,0h e bloqueando a edição manual (`min_value=1.0`).	Criação de `_parse_meta_dia()` preservando 0,0h, status *"Feriado / Abono"* e ajuste para `min_value=0.0`.
Linhas duplicadas em relatórios	Requisições simultâneas ao webhook podiam gerar duas linhas para a mesma data na planilha.	Implementação de `_deduplicate_records()` em `storage.py` e `drop_duplicates()` em `calculations.py`, garantindo unicidade por data.
Lentidão a cada clique na interface	O banco de dados era baixado da nuvem em toda re-execução do script Streamlit.	Cacheamento em `st.session_state.db` com atualização automática ao salvar e botão *"Sincronizar Nuvem"* na barra lateral.
Excesso de documentos na visão da chefia	Existência de dois modelos de folha (Espelho A4 genérico e Boletim FUSP).	Remoção do Espelho A4 genérico na visão da supervisão, mantendo exclusivamente o **Boletim de Frequência Oficial da FUSP**.

7. Guia Rápido de Operação e Credenciais

Dados de Acesso e Links Oficiais

- **Link Oficial (Nuvem):** https://ponto-estagio-carlos.streamlit.app
- **Link Direto para a Supervisora (Somente Leitura):** https://ponto-estagio-carlos.streamlit.app/?acesso=gestao
- **Usuário Padrão (Estagiário):** `carlos`
- **Senha Padrão Inicial:** `1122` *(pode ser alterada a qualquer momento na tela de login ou na barra lateral em "Configurações & Contrato", ficando salva na planilha da nuvem).*

7.1 Rotina Mensal de Fechamento FUSP

1. Registre normalmente suas entradas, pausas e saídas diariamente pelo painel **Registro Rápido**.
2. Caso esqueça alguma batida ou haja um feriado/recesso, utilize a aba **Ajuste Manual** (para feriados, basta definir a carga do dia como `0.0` e informar *"Feriado"* na observação).
3. Ao encerrar o expediente do **dia 20 de cada mês** (fechamento do ciclo FUSP 21/mês − 1 a 20/mês), acesse a aba **Boletim FUSP**.
4. Clique em **"BAIXAR BOLETIM FUSP PREENCHIDO (HTML / PDF)"** ou no botão **"Imprimir / Salvar como PDF"** dentro da prévia do documento para gerar o PDF oficial, colher as assinaturas e encaminhar até o dia 22.

Documento Técnico Gerado em LaTeX

Sistema de Ponto Eletrônico ICMC • USP — Outubro de 2026